savourai Version control Select Device <no device selected> main.dart

Project

- savourai ~/Desktop/MC Final Project/savourai
  - .dart_tool
  - .idea
  - android [savourai_android]
  - assets
  - ios
  - lib
    - models
    - screens
    - services
    - firebase_options.dart
    - main.dart
  - macos
  - test
  - .env
  - .flutter-plugins-dependencies
  - .gitignore
  - .metadata
  - analysis_options.yaml
  - firebase.json
  - LICENSE
  - pubspec.lock
  - pubspec.yaml
  - README.md
  - savourai.iml
- External Libraries
- Scratches and Consoles

main.dart

```dart
import ...

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await dotenv.load(fileName: ".env");

  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);

  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      theme: ThemeData(
        primarySwatch: Colors.blue,
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.teal),
        useMaterial3: true,
      ), ThemeData
      debugShowCheckedModeBanner: false,
      home: AuthWrapper(),
    ); MaterialApp
  }
}

class AuthWrapper extends StatelessWidget {
  const AuthWrapper({super.key});

  @override
  Widget build(BuildContext context) {
    return StreamBuilder<User?>(
      stream: FirebaseAuth.instance.authStateChanges(),
      builder: (context, snapshot) {
        if (snapshot.connectionState == ConnectionState.waiting) {
          return const Scaffold(
            body: Center(child: CircularProgressIndicator()),
          ); Scaffold
        }
        return snapshot.hasData ? Home() : LoginScreen();
      },
    ); StreamBuilder
  }
}
```

savourai > lib > main.dart 1:1 LF UTF-8 2 spaces

savourai | Version control | Select Device | <no device selected> | main.dart

Project

- savourai ~/Desktop/MC Final Project/savourai
  - .dart_tool
  - .idea
  - android [savourai_android]
  - assets
  - ios
  - lib
    - models
    - screens
    - services
    - firebase_options.dart
    - main.dart
  - macos
  - test
  - .env
  - .flutter-plugins-dependencies
  - .gitignore
  - .metadata
  - analysis_options.yaml
  - firebase.json
  - LICENSE
  - pubspec.lock
  - pubspec.yaml
  - README.md
  - savourai.iml
- External Libraries
- Scratches and Consoles

main.dart | firebase_options.dart

```dart
class DefaultFirebaseOptions {
  static FirebaseOptions get currentPlatform {
    if (kIsWeb) {
      return web;
    }
    switch (defaultTargetPlatform) {
      case TargetPlatform.android:
        return android;
      case TargetPlatform.iOS:
        return ios;
      case TargetPlatform.macOS:
        return macos;
      case TargetPlatform.windows:
        return windows;
      case TargetPlatform.linux:
        throw UnsupportedError(
          'DefaultFirebaseOptions have not been configured for linux - '
          'you can reconfigure this by running the FlutterFire CLI again.',
        );
      default:
        throw UnsupportedError(
          'DefaultFirebaseOptions are not supported for this platform.',
        );
    }
  }

  static const FirebaseOptions web = FirebaseOptions(
    apiKey: 'AIzaSyCWBCmNKRDfGhe2pM4-kkFPitk0mF2korE',
    appId: '1:567633879182:web:3be81a6dcd603f1c87e9df',
    messagingSenderId: '567633879182',
    projectId: 'savour-ai-app',
    authDomain: 'savour-ai-app.firebaseapp.com',
    storageBucket: 'savour-ai-app.firebasestorage.app',
    measurementId: 'G-7PEBNYX1G8',
  );

  static const FirebaseOptions android = FirebaseOptions(
    apiKey: 'AIzaSyDwVaXyrmEaxA5_t2jVLRcgbFRRhudXgPQ',
    appId: '1:567633879182:android:b6c37c25b65bb04987e9df',
    messagingSenderId: '567633879182',
    projectId: 'savour-ai-app',
    storageBucket: 'savour-ai-app.firebasestorage.app',
  );

  static const FirebaseOptions ios = FirebaseOptions(
    apiKey: 'AIzaSyCH6phWxTb3rfTSQbPrOHnWS88H-DBqsXY',
    appId: '1:567633879182:ios:73027dba38b5c2e087e9df',
    messagingSenderId: '567633879182',
    projectId: 'savour-ai-app',
    storageBucket: 'savour-ai-app.firebasestorage.app',
```

29

savourai > lib > firebase_options.dart | 1:1 | LF | UTF-8 | 2 spaces

savourai | Version control | Select Device | <no device selected> | main.dart

Project

- savourai ~/Desktop/MC Final Project/savourai
  - .dart_tool
  - .idea
  - android [savourai_android]
  - assets
  - ios
  - lib
    - models
    - screens
    - services
      - chatservice.dart
      - connectivityservice.dart
      - favourites_service.dart
    - firebase_options.dart
    - main.dart
  - macos
  - test
  - .env
  - .flutter-plugins-dependencies
  - .gitignore
  - .metadata
  - analysis_options.yaml
  - firebase.json
  - LICENSE
  - pubspec.lock
  - pubspec.yaml
  - README.md
  - savourai.iml
- External Libraries
- Scratches and Consoles

main.dart | firebase_options.dart | chatservice.dart

2 28 42

```
**Here's what I can help you with:**
• 🍳 **Cooking questions** - recipes, techniques, troubleshooting
• 💬 **Casual conversation** - greetings, jokes, general chat
• 💡 **Creative ideas** - meal planning, flavor combinations
• 🆘 **Problem solving** - kitchen disasters, substitutions

**Try asking me:**
• "How do I make [dish name]?"
• "What can I substitute for [ingredient]?"
• "Tell me a joke!"
• "What should I cook for dinner?"
• "How are you today?"

Or just tell me what's on your mind! I'm here to help and chat. 😊

By the way, have you tried making pasta from scratch? It's easier than you think!""";
  }

  String _getQuickResponse(String message) {
    final lower = message.toLowerCase();
    final responses = [
      "That's interesting! Tell me more about what you're thinking. 😊",
      "I'd love to help with that! What specifically are you wondering about?",
      "Great question! I'm here to chat about cooking or anything else you'd like.",
      "Thanks for sharing! I'm always excited to help people cook and learn.",
      "Let's talk about that! I'm all ears (well, metaphorically speaking!).",
      "Interesting point! Want to dive deeper into that topic?",
    ];

    // Match some common patterns
    if (lower.contains("?")) return "That's a great question! Let me think about that...";
    if (lower.contains("!")) return "Exciting! Tell me more!";

    return responses[Random().nextInt(responses.length)];
  }

  String _getCookingTip(String message) {
    final lower = message.toLowerCase();
    if (lower.contains("pasta")) return "Remember: Salt your pasta water until it tastes like the sea!";
    if (lower.contains("chicken")) return "Pro tip: Let chicken rest 5 minutes after cooking for juicier meat.";
    if (lower.contains("rice")) return "Perfect rice secret: Don't peek while it's cooking!";
    return "Always taste your food as you cook - seasoning makes all the difference!";
  }
}

// Helper function
int min(int a, int b) => a < b ? a : b;
```

savourai > lib > services > chatservice.dart | 391:40 | LF | UTF-8 | 2 spaces

savourai | Version control | Select Device | <no device selected> | main.dart

Project

- savourai ~/Desktop/MC Final Project/savourai
  - .dart_tool
  - .idea
  - android [savourai_android]
  - assets
  - ios
  - lib
    - models
    - screens
    - services
      - chatservice.dart
      - connectivityservice.dart
      - favourites_service.dart
    - firebase_options.dart
    - main.dart
  - macos
  - test
  - .env
  - .flutter-plugins-dependencies
  - .gitignore
  - .metadata
  - analysis_options.yaml
  - firebase.json
  - LICENSE
  - pubspec.lock
  - pubspec.yaml
  - README.md
  - savourai.iml
- External Libraries
- Scratches and Consoles

main.dart | firebase_options.dart | chatservice.dart | connectivityservice.dart

```dart
import ...

class ConnectivityService {
  final Connectivity _connectivity = Connectivity();
  bool _isDialogShowing = false;

  Future<void> checkConnectivity(BuildContext context) async {
    List<ConnectivityResult> results = await _connectivity.checkConnectivity();
    for (var result in results) {
      _showConnectivityStatus(context, result);
    }

    _connectivity.onConnectivityChanged.listen((
      List<ConnectivityResult> results,
    ) {
      for (var result in results) {
        _showConnectivityStatus(context, result);
      }
    });
  }

  void _showConnectivityStatus(
    BuildContext context,
    ConnectivityResult result,
  ) {
    if (result == ConnectivityResult.none) {
      if (!_isDialogShowing) {
        _showNoConnectionDialog(context);
      }
    } else {
      if (_isDialogShowing) {
        _isDialogShowing = false;
        Navigator.of(context, rootNavigator: true).pop();
      }
    }
  }

  void _showNoConnectionDialog(BuildContext context) {
    _isDialogShowing = true;

    showDialog(
      context: context,
      barrierDismissible: false,
      builder: (BuildContext context) {
        return Container(
          child: LottieBuilder.asset('assets/noconnection.json'),
        );  Container
      },
    );
```

savourai > lib > services > connectivityservice.dart | 1:1 | LF | UTF-8 | 2 spaces

savourai | Version control | Select Device | <no device selected> | main.dart

Project

- savourai ~/Desktop/MC Final Project/savourai
  - .dart_tool
  - .idea
  - android [savourai_android]
  - assets
  - ios
  - lib
    - models
    - screens
    - services
      - chatservice.dart
      - connectivityservice.dart
      - favourites_service.dart
    - firebase_options.dart
    - main.dart
  - macos
  - test
  - .env
  - .flutter-plugins-dependencies
  - .gitignore
  - .metadata
  - analysis_options.yaml
  - firebase.json
  - LICENSE
  - pubspec.lock
  - pubspec.yaml
  - README.md
  - savourai.iml
- External Libraries
- Scratches and Consoles

main.dart | firebase_options.dart | chatservice.dart | connectivityservice.dart | favourites_service.dart

34 3

```dart
import ...

class FavoritesService {
  final FirebaseFirestore _firestore = FirebaseFirestore.instance;
  final User? _user = FirebaseAuth.instance.currentUser;

  Future<void> saveMeal(Meal meal) async {
    if (_user == null) {
      print('❌ Cannot save meal: User not logged in');
      return;
    }

    try {
      print('💾 Saving meal to Firestore: ${meal.name} (ID: ${meal.id})');
      print('   Thumbnail URL: ${meal.thumbnail}');

      await _firestore
          .collection('users')
          .doc(_user.uid)
          .collection('saved')
          .doc(meal.id)
          .set(meal.toJson(), SetOptions(merge: true));

      print('✅ Meal saved successfully!');
    } catch (e) {
      print('❌ Error saving meal: $e');
      rethrow;
    }
  }

  Future<void> addToFavorites(Meal meal) async {
    if (_user == null) {
      print('❌ Cannot add favorite: User not logged in');
      return;
    }

    try {
      print('❤️ Adding to favorites: ${meal.name} (ID: ${meal.id})');

      await _firestore
          .collection('users')
          .doc(_user.uid)
          .collection('favorites')
          .doc(meal.id)
          .set(meal.toJson(), SetOptions(merge: true));

      print('✅ Added to favorites successfully!');
    } catch (e) {
      print('❌ Error adding to favorites: $e');
```

savourai > lib > services > favourites_service.dart | 1:1 | LF | UTF-8 | 2 spaces

savourai | Version control | Select Device | <no device selected> | main.dart

Project

- savourai ~/Desktop/MC Final Project/savourai
  - .dart_tool
  - .idea
  - android [savourai_android]
  - assets
  - ios
  - lib
    - models
    - screens
      - account.dart
      - chat.dart
      - feed.dart
      - home.dart
      - login.dart
      - mealpage.dart
      - search.dart
    - services
    - firebase_options.dart
    - main.dart
  - macos
  - test
  - .env
  - .flutter-plugins-dependencies
  - .gitignore
  - .metadata
  - analysis_options.yaml
  - firebase.json
  - LICENSE
  - pubspec.lock
  - pubspec.yaml
  - README.md
  - savourai.iml
- External Libraries

firebase_options.dart | chatservice.dart | connectivityservice.dart | favourites_service.dart | account.dart

5 13 8

```
import ...

class AccountScreen extends StatefulWidget {
  const AccountScreen({Key? key}) : super(key: key);

  @override
  State<AccountScreen> createState() => _AccountScreenState();
}

class _AccountScreenState extends State<AccountScreen> {
  final User? user = FirebaseAuth.instance.currentUser;
  final FirebaseFirestore _firestore = FirebaseFirestore.instance;
  final FavoritesService _favoritesService = FavoritesService();
  List<Meal> savedMeals = [];
  List<Meal> favoriteMeals = [];
  bool isLoading = true;
  String _appVersion = "";
  bool _showAbout = false;
  String? _userName; // Store user's name

  @override
  void initState() {
    super.initState();
    _loadUserData();
    _loadAppInfo();
    _fetchUserName(); // Fetch user's name
  }

  // NEW: Fetch user's name from Firestore
  Future<void> _fetchUserName() async {
    if (user == null) {
      setState(() => _userName = null);
      return;
    }

    try {
      final userDoc = await _firestore
          .collection('users')
          .doc(user!.uid)
          .get();

      if (userDoc.exists) {
        setState(() {
          _userName = userDoc.data()?['name'];
        });
      } else {
        // Fallback to Firebase Auth display name
        setState(() {
          _userName = user?.displayName;
```

savourai > lib > screens > account.dart | 515:20 | LF | UTF-8 | 2 spaces

savourai Version control Select Device <no device selected> main.dart

Project

- savourai ~/Desktop/MC Final Project/savourai
  - .dart_tool
  - .idea
  - android [savourai_android]
  - assets
  - ios
  - lib
    - models
    - screens
      - account.dart
      - chat.dart
      - feed.dart
      - home.dart
      - login.dart
      - mealpage.dart
      - search.dart
    - services
    - firebase_options.dart
    - main.dart
  - macos
  - test
  - .env
  - .flutter-plugins-dependencies
  - .gitignore
  - .metadata
  - analysis_options.yaml
  - firebase.json
  - LICENSE
  - pubspec.lock
  - pubspec.yaml
  - README.md
  - savourai.iml
- External Libraries

chatservice.dart | connectivityservice.dart | favourites_service.dart | account.dart | feed.dart | chat.dart

```dart
import ...

class Chat extends StatefulWidget {
  const Chat({super.key});

  @override
  State<Chat> createState() => _ChatState();
}

class _ChatState extends State<Chat> {
  List<Map<String, dynamic>> messages = [];
  TextEditingController controller = TextEditingController();
  bool _isLoading = false;

  @override
  void initState() {
    super.initState();
    messages.clear();
    // Optional: Send welcome message
    // chatFn("Hello! Give a cooking tip");
  }

  chatFn(String chatMsg) async {
    if (chatMsg.trim().isEmpty) return;

    controller.clear();
    setState(() {
      _isLoading = true;
      messages.add({"role": "user", "content": chatMsg.trim()});
    });

    try {
      // This should call ChefBotService
      var chat = await ChefBotService().sendMessage(chatMsg);

      await Future.delayed(Duration(milliseconds: 500));

      setState(() {
        messages.add({"role": "system", "content": chat});
        _isLoading = false;
      });
    } catch (e) {
      print("Chat error: $e");
      setState(() {
        messages.add({
          "role": "system",
          "content": "The chef is thinking... Try asking about pasta, chicken, or cooking tips!"
        });
        _isLoading = false;
```

savourai > lib > screens > chat.dart 286:2 LF UTF-8 2 spaces

savourai · Version control · Select Device · <no device selected> · main.dart

Project

- savourai ~/Desktop/MC Final Project/savourai
  - .dart_tool
  - .idea
  - android [savourai_android]
  - assets
  - ios
  - lib
    - models
    - screens
      - account.dart
      - chat.dart
      - feed.dart
      - home.dart
      - login.dart
      - mealpage.dart
      - search.dart
    - services
    - firebase_options.dart
    - main.dart
  - macos
  - test
  - .env
  - .flutter-plugins-dependencies
  - .gitignore
  - .metadata
  - analysis_options.yaml
  - firebase.json
  - LICENSE
  - pubspec.lock
  - pubspec.yaml
  - README.md
  - savourai.iml
- External Libraries

chatservice.dart | connectivityservice.dart | favourites_service.dart | account.dart | feed.dart | chat.dart

```dart
import ...

class Feed extends StatefulWidget {
  const Feed({super.key});

  @override
  State<Feed> createState() => _FeedState();
}

class _FeedState extends State<Feed> {
  final PageController pageController = PageController();
  final List<Meal> meals = [];
  final List<Color> colors = [];
  bool loading = false;

  @override
  void initState() {
    super.initState();
    loadMoreMeals();

    pageController.addListener(() {
      if (pageController.page != null &&
          pageController.page!.round() >= meals.length - 1 &&
          !loading) {
        loadMoreMeals();
      }
    });
  }

  @override
  void dispose() {
    pageController.dispose();
    super.dispose();
  }


  Future<void> getColorsFromImage(String imageUrl) async {
    try {
      final palette = await PaletteGenerator.fromImageProvider(
        NetworkImage(imageUrl),
      );
      colors.add(palette.lightVibrantColor?.color ?? const Color(0xFFFFDD6E));
    } catch (e) {
      colors.add(const Color(0xFFFFDD6E));
    }
  }

  Future<void> loadMoreMeals() async {
    setState(() => loading = true);
```

savourai > lib > screens > feed.dart · 1:1 · LF · UTF-8 · 2 spaces

savourai | Version control | Select Device | <no device selected> | main.dart

Project

- savourai ~/Desktop/MC Final Project/savourai
  - .dart_tool
  - .idea
  - android [savourai_android]
  - assets
  - ios
  - lib
    - models
    - screens
      - account.dart
      - chat.dart
      - feed.dart
      - home.dart
      - login.dart
      - mealpage.dart
      - search.dart
    - services
    - firebase_options.dart
    - main.dart
  - macos
  - test
  - .env
  - .flutter-plugins-dependencies
  - .gitignore
  - .metadata
  - analysis_options.yaml
  - firebase.json
  - LICENSE
  - pubspec.lock
  - pubspec.yaml
  - README.md
  - savourai.iml
- External Libraries

chatservice.dart | connectivityservice.dart | favourites_service.dart | account.dart | feed.dart | home.dart

6 9 10

```dart
import 'package:flutter/material.dart';
import 'package:google_fonts/google_fonts.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';
import 'package:savourai/models/meal.dart';
import 'package:savourai/screens/chat.dart';
import 'package:savourai/screens/mealpage.dart';
import 'package:savourai/screens/search.dart';
import 'package:savourai/screens/account.dart';
import 'package:savourai/services/favourites_service.dart';
import 'package:crystal_navigation_bar/crystal_navigation_bar.dart';
import 'package:iconly/iconly.dart';

class Home extends StatefulWidget {
  const Home({super.key});

  @override
  State<Home> createState() => _HomeState();
}

class _HomeState extends State<Home> {
  int selectedIndex = 0;
  Meal? recipeOfTheDay;
  List<Meal> randomRecipes = [];
  bool isLoading = true;
  bool _loadingMore = false;
  bool _hasMoreRecipes = true;
  int _currentPage = 0;
  final int _recipesPerPage = 10;
  final FavoritesService _favoritesService = FavoritesService();
  final User? user = FirebaseAuth.instance.currentUser;
  final FirebaseFirestore _firestore = FirebaseFirestore.instance;
  String? _userName; // Store user's name
  final ScrollController _scrollController = ScrollController();
  int _lastSelectedIndex = 0;

  @override
  void initState() {
    super.initState();
    _loadInitialRecipes();
    _fetchUserName(); // Fetch user's name

    // Add scroll listener for infinite scrolling
    _scrollController.addListener(_scrollListener);
  }

  @override
```

savourai > lib > screens > home.dart

253:52 (12 chars) LF UTF-8 2 spaces

savourai Version control Select Device <no device selected> main.dart

Project

- savourai ~/Desktop/MC Final Project/savourai
  - .dart_tool
  - .idea
  - android [savourai_android]
  - assets
  - ios
  - lib
    - models
    - screens
      - account.dart
      - chat.dart
      - feed.dart
      - home.dart
      - login.dart
      - mealpage.dart
      - search.dart
    - services
    - firebase_options.dart
    - main.dart
  - macos
  - test
  - .env
  - .flutter-plugins-dependencies
  - .gitignore
  - .metadata
  - analysis_options.yaml
  - firebase.json
  - LICENSE
  - pubspec.lock
  - pubspec.yaml
  - README.md
  - savourai.iml
- External Libraries

connectivityservice.dart | favourites_service.dart | account.dart | feed.dart | home.dart | login.dart

```dart
import ...

class LoginScreen extends StatefulWidget {
  const LoginScreen({Key? key}) : super(key: key);

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final TextEditingController _emailController = TextEditingController();
  final TextEditingController _passwordController = TextEditingController();
  final TextEditingController _nameController = TextEditingController(); // Added for name

  bool _isLoading = false;
  bool _isSignUp = false;

  Future<void> _signInWithEmail() async {
    // Validate fields based on mode
    if (_isSignUp) {
      // Signup validation - check all fields
      if (_emailController.text.isEmpty ||
          _passwordController.text.isEmpty ||
          _nameController.text.isEmpty) {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text('Please fill all fields')),
        );
        return;
      }
    } else {
      // Login validation - only email and password
      if (_emailController.text.isEmpty || _passwordController.text.isEmpty) {
        ScaffoldMessenger.of(context).showSnackBar(
          const SnackBar(content: Text('Please fill all fields')),
        );
        return;
      }
    }

    setState(() => _isLoading = true);
    try {
      if (_isSignUp) {
        // 1. Create user in Firebase Auth
        UserCredential userCredential =
        await FirebaseAuth.instance.createUserWithEmailAndPassword(
          email: _emailController.text.trim(),
          password: _passwordController.text.trim(),
        );
```

savourai > lib > screens > login.dart 309:2 LF UTF-8 2 spaces

savourai | Version control | Select Device | <no device selected> | main.dart

Project

- savourai ~/Desktop/MC Final Project/savourai
  - .dart_tool
  - .idea
  - android [savourai_android]
  - assets
  - ios
  - lib
    - models
    - screens
      - account.dart
      - chat.dart
      - feed.dart
      - home.dart
      - login.dart
      - mealpage.dart
      - search.dart
    - services
    - firebase_options.dart
    - main.dart
  - macos
  - test
  - .env
  - .flutter-plugins-dependencies
  - .gitignore
  - .metadata
  - analysis_options.yaml
  - firebase.json
  - LICENSE
  - pubspec.lock
  - pubspec.yaml
  - README.md
  - savourai.iml
- External Libraries

vice.dart | favourites_service.dart | account.dart | feed.dart | home.dart | login.dart | mealpage.dart

```dart
import ...

class MealPage extends StatefulWidget {
  final Meal meal;
  final Color backgroundColor;
  const MealPage({Key? key, required this.meal, required this.backgroundColor})
      : super(key: key);

  @override
  State<MealPage> createState() => _MealPageState();
}

class _MealPageState extends State<MealPage> {
  bool isFavorite = false;
  final FavoritesService _favoritesService = FavoritesService();

  @override
  void initState() {
    super.initState();
    _checkIfFavorite();
  }

  Future<void> _checkIfFavorite() async {
    final favorite = await _favoritesService.isMealFavorited(widget.meal.id);
    setState(() {
      isFavorite = favorite;
    });
  }

  Future<void> _toggleFavorite() async {
    try {
      if (isFavorite) {
        await _favoritesService.removeFromFavorites(widget.meal.id);
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(
            content: Text('Removed from favorites'),
            backgroundColor: Colors.orange,
          ), SnackBar
        );
      } else {
        await _favoritesService.addToFavorites(widget.meal);
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(
            content: Text('Added to favorites!'),
            backgroundColor: Colors.green,
          ), SnackBar
        );
      }
      // Update the UI state
```

savourai > lib > screens > mealpage.dart | 266:40 | LF | UTF-8 | 2 spaces

savourai · Version control · Select Device · <no device selected> · main.dart

Project

- savourai ~/Desktop/MC Final Project/savourai
  - .dart_tool
  - .idea
  - android [savourai_android]
  - assets
  - ios
  - lib
    - models
    - screens
      - account.dart
      - chat.dart
      - feed.dart
      - home.dart
      - login.dart
      - mealpage.dart
      - search.dart
    - services
    - firebase_options.dart
    - main.dart
  - macos
  - test
  - .env
  - .flutter-plugins-dependencies
  - .gitignore
  - .metadata
  - analysis_options.yaml
  - firebase.json
  - LICENSE
  - pubspec.lock
  - pubspec.yaml
  - README.md
  - savourai.iml
- External Libraries

ourites_service.dart | account.dart | feed.dart | home.dart | login.dart | mealpage.dart | search.dart

2 4 3

```dart
import ...

class Search extends StatefulWidget {
  const Search({super.key});

  @override
  State<Search> createState() => _SearchState();
}

class _SearchState extends State<Search> {
  final List<String> recipes = [
    "Chicken",
    "Spaghetti",
    "Cake",
    "Stew",
    "Pork",
    "Tofu",
    "Tuna",
    "Salad",
  ];
  List<Meal> searchedResult = [];
  Future<Meal?> fetchMeal(String recipe) async {
    final url = 'https://www.themealdb.com/api/json/v1/1/search.php?s=$recipe';
    try {
      final response = await http.get(Uri.parse(url));
      if (response.statusCode == 200) {
        final data = json.decode(response.body);
        final mealsList = data['meals'] as List<dynamic>;

        if (mealsList.isNotEmpty) {
          for (int i = 0; i < mealsList.length; i++) {
            final meal = Meal.fromJson(mealsList[i]);
            searchedResult.add(meal);
          }
        }
      }
    } catch (e) {
      print('Error fetching meal: $e');
    }

    setState(() {});
    return null;
  }

  openMeal(String id) async {
    final url = 'https://www.themealdb.com/api/json/v1/1/lookup.php?i=$id';
    try {
      final response = await http.get(Uri.parse(url));
      if (response.statusCode == 200) {
```

savourai > lib > screens > search.dart · 16:1 · LF · UTF-8 · 2 spaces

savourai Version control Select Device <no device selected> main.dart

Project

- savourai ~/Desktop/MC Final Project/savourai
  - .dart_tool
  - .idea
  - android [savourai_android]
  - assets
  - ios
  - lib
    - models
      - meal.dart
    - screens
    - services
    - firebase_options.dart
    - main.dart
  - macos
  - test
  - .env
  - .flutter-plugins-dependencies
  - .gitignore
  - .metadata
  - analysis_options.yaml
  - firebase.json
  - LICENSE
  - pubspec.lock
  - pubspec.yaml
  - README.md
  - savourai.iml
- External Libraries
- Scratches and Consoles

account.dart feed.dart home.dart login.dart mealpage.dart search.dart meal.dart

```dart
import 'package:cloud_firestore/cloud_firestore.dart';

class Meal {
  final String id;
  final String name;
  final String category;
  final String area;
  final String instructions;
  final String thumbnail;
  final String? youtubeUrl;
  final List<Map<String, String>> ingredients;

  Meal({
    required this.id,
    required this.name,
    required this.category,
    required this.area,
    required this.instructions,
    required this.thumbnail,
    this.youtubeUrl,
    required this.ingredients,
  });

  // Factory constructor for parsing API JSON (from TheMealDB)
  factory Meal.fromJson(Map<String, dynamic> json) {
    List<Map<String, String>> ingredients = [];

    for (int i = 1; i <= 20; i++) {
      final ingredient = json['strIngredient$i'];
      final measure = json['strMeasure$i'];

      if (ingredient != null &&
          ingredient.toString().trim().isNotEmpty &&
          measure != null &&
          measure.toString().trim().isNotEmpty) {
        ingredients.add({
          'ingredient': ingredient.toString().trim(),
          'measure': measure.toString().trim(),
        });
      }
    }

    return Meal(
      id: json['idMeal'] ?? '',
      name: json['strMeal'] ?? '',
      category: json['strCategory'] ?? '',
      area: json['strArea'] ?? '',
      instructions: json['strInstructions'] ?? '',
      thumbnail: json['strMealThumb'] ?? '',
```

savourai > lib > models > meal.dart 1:1 LF UTF-8 2 spaces

savourai · Version control · Select Device · <no device selected> · main.dart

Project

- savourai ~/Desktop/MC Final Project/savourai
  - .dart_tool
  - .idea
  - android [savourai_android]
  - assets
  - ios
  - lib
  - macos
  - test
  - .env
  - .flutter-plugins-dependencies
  - .gitignore
  - .metadata
  - analysis_options.yaml
  - firebase.json
  - LICENSE
  - pubspec.lock
  - pubspec.yaml
  - README.md
  - savourai.iml
- External Libraries
- Scratches and Consoles

feed.dart | home.dart | login.dart | mealpage.dart | search.dart | meal.dart | pubspec.yaml

Flutter commands: Pub get | Pub upgrade | Pub outdated | Flutter doctor

```yaml
  sdk: '>=3.7.2 <4.0.0'

dependencies:
  flutter:
    sdk: flutter

  # UI & Design
  cupertino_icons: ^1.0.8
  crystal_navigation_bar: ^1.0.4
  iconly: ^1.0.1
  glass: ^2.0.0+2
  lottie: ^3.3.1
  google_fonts: ^6.2.1
  flutter_animate: ^4.5.2

  # Networking & API
  http: ^1.4.0
  cached_network_image: ^3.4.1

  # Firebase
  firebase_core: ^2.24.2
  firebase_auth: ^4.12.5
  cloud_firestore: ^4.13.1
  google_sign_in: ^6.1.5

  # Utilities
  url_launcher: ^6.3.1
  package_info_plus: ^8.3.0
  palette_generator: ^0.3.3+7
  flutter_dotenv: ^5.2.1
  connectivity_plus: ^6.1.4

  # Chat & Search
  flutter_markdown: ^0.7.7+1
  advanced_search: ^2.2.5

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^5.0.0
  flutter_launcher_icons: ^0.14.4

flutter_launcher_icons:
  android: "launcher_icon"
  ios: true
  image_path: "assets/savourAppLogo.png"
  min_sdk_android: 21
```

savourai > pubspec.yaml · 1:1 · LF · UTF-8 · 2 spaces · No JSON schema